Scientific Computing on Hardware

Subhodeep Chakraborty, Krishna Patil, Nara Prajwal and G.V.V. Sharma
Department of Electrical Engineering
IIT Hyderabad
Email: gadepall@ee.iith.ac.in

***Abstract*—In this paper, various algebraic and transcendental functions are implemented on microcontroller hardware. This is done by first modeling most functions using high school level differential equations, converting them to difference equations using elementary numerical techniques and then porting these algorithms to an ATMEGA328P microcontroller using AVR-GCC. In the process, we compare different numerical methods and finalize the Runge-Kutta (RK4) method for hardware implementation, based on numerical accuracy. We demonstrate the practical utility of this approach by building a scientific calculator based on the RK4 algorithm.**

***Index Terms*—Scientific Calculator, Microcontroller, Runge-Kutta Method, Quake III Algorithm, Shunting Yard Algorithm, Embedded C.**

I. Introduction

Physical systems are typically modeled as a setup of ordinary differential equations (ODE) where the gradient of a state variable x is time and state dependent: $\frac{dx}{dt} = f(x,t)$, with an initial condition given as $x(t_0) = x_0$. Most physical systems are now being implemented on digital platforms. This requires the design and implementation of algorithms capable of approximating complex functions under tight resource constraints. With respect to the ubiquity of differential equations as descriptors of real-world systems, the algorithms under the lens are those that perform or approximate numerical integration [1].

Numerical methods for function approximation on hardware often prioritize either simplicity or specialized efficiency. Euler's method is the simplest to program, yet its first-order accuracy ($O(h)$) leads to significant numerical drift and instability unless excessively small step sizes are used [2]. Second-order Runge-Kutta (RK2) methods improve precision but still feature much greater absolute error magnitudes than RK4 [3].

Alternative approximation strategies such as Taylor series or Maclaurin expansions are geometrically intuitive but often impractical for embedded systems [4]. This is because these methods require either the tedious recursive calculation of high-order derivatives, which are computationally expensive to evaluate, or for the coefficients for each function to be pre-stored, which results in memory usage piling up quickly for acceptable accuracy [1]. Also, the number of terms, and thus calculations needed for convergence, are much higher for Taylor series as compared to RK4 for the same accuracy [5]. In contrast, the RK4 framework achieves high-order accuracy without needing derivatives, instead evaluating the function at four strategically chosen intermediate points. This allows for a more robust and maintainable implementation when derivatives are complex or unavailable [4].

For specific transcendental functions, the CORDIC algorithm is highly efficient as it uses simple shift-and-add operations instead of multiplications. However, CORDIC is limited by specific convergence ranges and requires different recursive sequences for different classes of functions [6]. Linear multistep methods like Adams-Bashforth can reduce function evaluations but are not self-starting and necessitate a queue of past history to be stored that increases memory usage [2].

More advanced structural equations and orthogonal collocation methods offer low levels of error but frequently involve complex and heavy computations or iterative nonlinear solvers [7]. Similarly, adaptive methods like ode45 (used in MATLAB) provide excellent accuracy but introduce variable execution times that can cause unacceptable jitter in real-time environments, such as the those mentioned above [8].

A fixed-step RK4 approach is superior for predictable performance under the constraints discussed in this work, ensuring that calculations complete reliably within strict sampling windows, such as those in robotics or flight controllers [4]. Further, the single-step nature of RK4 is more suitable for limited microcontrollers as it avoids the RAM overhead of managing past solution data.

Despite extensive research into specialized hardware like FPGAs or complex applications such as GPS orbit computation and motor control, there is a lack of a unified framework for general scientific computing on constrained 8-bit platforms [8], [9]. Most related works in embedded systems treat different transcendental functions as separate implementation problems rather than a single unified mathematical structure [10].

This work bridges that gap by demonstrating how a numerical computing concept, the reformulation of diverse scientific functions into high school-level ODEs, can be used as a unified, fixed-step software architecture. This provides a robust, high-precision, and conceptually accessible pedagogical and practical framework for relatively limited microcontrollers. The proposed algorithms are then validated by implementing a scientific calculator using the ATMEGA328P.

II. Calculator: Software Implementation

We first test the numerical algorithms in software before proceeding to hardware. Table I lists the functions imple-

mented on the calculator while Table II lists the functions that can be formulated through ODEs [11].

TABLE I: Supported Functions and Operations

Category	**Functions / Operations**
Trigonometric	$\sin(x), \tan(x), \cos(x)$
Inv. Trig.	$\tan^{-1}(x), \cos^{-1}(x), \sin^{-1}(x)$
Exp. / Logarithmic	$e^x, \ln(x)$
Power / Root	x^y, $\sqrt{x}$
Factorial	$n!$
Basic Arithmetic	$+, -, \times, \div$
Constants	π, e
Input Symbols	Digits 0–9, decimal point, parentheses (,)

TABLE II: Implemented Functions and their ODEs

Function $y(x)$	**Differential Equation**	**Initial Conditions**
$\sin(x)$	$y'' + y = 0$	$y(0) = 0, \quad y'(0) = 1$
$\sin^{-1}(x)$	$\sqrt{(1-x^2)}y' - 1 = 0$	$y(0) = 0$
$\tan^{-1}(x)$	$(1+x^2)y' - 1 = 0$	$y(0) = 0$
$\ln(x)$	$xy' - 1 = 0$	$y(1) = 0$
e^x	$y' - y = 0$	$y(0) = 1$
x^w	$xy' - wy = 0$	$y(1) = 1$

A. RK4 based functions

The second-order differential equation for the sine function listed in Table II can be expressed as

$$\frac{dy}{dx} = z, \quad \frac{dz}{dx} = -y, \quad y(0) = 0,\ z(0) = 1 \tag{1}$$

The RK4 method then yields the update equation (see Appendix A-A).

$$\begin{aligned} k_1 &= hz_n, & l_1 &= -hy_n \\ k_2 &= h\left(z_n + \tfrac{l_1}{2}\right), & l_2 &= -h\left(y_n + \tfrac{k_1}{2}\right) \\ k_3 &= h\left(z_n + \tfrac{l_2}{2}\right), & l_3 &= -h\left(y_n + \tfrac{k_2}{2}\right) \\ k_4 &= h(z_n + l_3), & l_4 &= -h(y_n + k_3) \\ y_{n+1} &= y_n + \tfrac{1}{6}(k_1 + 2k_2 & z_{n+1} &= z_n + \tfrac{1}{6}(l_1 + 2l_2 \\ &\quad +2k_3 + k_4) & &\quad +2l_3 + l_4) \end{aligned} \tag{2}$$

where k_1, k_2, k_3, k_4 correspond to the RK-4 terms of the equation $\frac{dy}{dx} = z$ and l_1, l_2, l_3, l_4 corresponds to the RK-4 terms of the equation $\frac{dz}{dx} = -y$. To clarify how first-order differential equations are solved using RK4, the example of the natural logarithm can be taken:

$$\frac{dy}{dx} = \frac{1}{x}, \quad y(1) = 0 \tag{3}$$

The RK4 update equation is obtained through simple algebraic substitution:

$$m_1 = h \cdot f(x_n, y_n) = \frac{h}{x_n} \tag{4}$$

$$m_2 = h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{m_1}{2}\right) = \frac{h}{x_n + \frac{h}{2}} \tag{5}$$

$$m_3 = h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{m_2}{2}\right) = \frac{h}{x_n + \frac{h}{2}} \tag{6}$$

$$m_4 = h \cdot f(x_n + h, y_n + m_3) = \frac{h}{x_n + h} \tag{7}$$

The remaining functions in Table II are computed using a similar approach in Table III. The other functions/constants are computed using standard equations [11]

$$\begin{aligned} \cos x = \sin\left(\frac{\pi}{2} - x\right), \tan x = \frac{\sin x}{\cos x}, \\ \cos^{-1}(x) = \frac{\pi}{2} - \sin^{-1}(x), \pi = 4\tan^{-1}(1) \end{aligned} \tag{8}$$

The reason for choosing RK4 over other numerical techniques is discussed in some detail in Appendix A-B

B. Non-RK4 Functions

1) The factorial is a simple recursive equation [12]

$$y_n = ny_{n-1} \tag{9}$$

2) The fast inverse square root algorithm (Quake III method) is used to efficiently compute $1/\sqrt{x}$ which is then used for computing $\sqrt{x}$ [13].

C. Expression Parsing

The Shunting Yard algorithm is employed for converting infix expressions into postfix form (Reverse Polish Notation, RPN) for evaluation [14]. It relies on two structures

- A stack for operators/functions
- An output queue for the final postfix expression

The Shunting Yard Algorithm, together with RPN evaluation, respects operator precedence, associativity, and bracket handling, making it a cornerstone in expression processing across computing applications. Instructions for installation of a software based prototype for testing the algorithms are given in Appendix B. These instructions are for an Android phone running termux-debian in proot.

III. Calculator: Hardware Implementation

Table IV lists the components required for the scientific calculator hardware. The Arduino-LCD connections are available in Table VII. The calculator operates in two modes. The set of functions and the respective keys for the two modes are listed in Tables V and VI. The schematic for circuit connections is available in Fig. 2 and the instructions for uploading the codes are available in Appendix C .

Function	k_1	k_2	k_3	k_4	y_{n+1}
Arcsine $\sin^{-1}(x)$	$\frac{h}{\sqrt{1-x_n^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+\frac{h}{2})^2}}$	$\frac{h}{\sqrt{1-(x_n+h)^2}}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Arctangent $\tan^{-1}(x)$	$\frac{h}{1+x_n^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+\frac{h}{2})^2}$	$\frac{h}{1+(x_n+h)^2}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Natural Log $\ln(x)$	$\frac{h}{x_n}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+\frac{h}{2}}$	$\frac{h}{x_n+h}$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
Power $y = x^w$	$h \cdot \frac{w}{x_n} y_n$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_1}{2}\right)$	$h \cdot \frac{w}{x_n+\frac{h}{2}} \left(y_n + \frac{k_2}{2}\right)$	$h \cdot \frac{w}{x_n+h} (y_n + k_3)$	$y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$

TABLE III: RK4 Update Equations for Single-Variable Functions

A. Button Matrix

The button matrix is a grid of push-buttons arranged in rows and columns. It allows multiple buttons to be connected to the microcontroller using fewer pins.

- The Arduino scans the matrix by activating each row (setting it LOW) one at a time while reading the columns.
- When a button is pressed, it completes the circuit.
- By identifying the active row and column, the Arduino determines which button was pressed.
- Example: If second row is pressed when first column is active, the first column's second button is pressed.

This reduces the number of pins required to implement a calculator with 25 buttons.

TABLE IV: Scientific calculator components

Qty	**Component**	**Description**
25	Pushbuttons	Used to form a button matrix for user input.
1	LCD 16×2	Connected to the Arduino for showing results.
1	Arduino Uno	Microcontroller to process inputs and execute calculations.
–	Jumper wires	Used for connections as shown in Fig. 2.
1	Potentiometer	Used for LCD contrast adjustment.

TABLE V: Key assignments in Mode 1 with Arduino pin mapping

Column/Row	1	2	3	4	5
1	0	1	2	3	4
2	5	6	7	8	9
3	+	-	×	÷	$\sin(x)$
4	$\cos(x)$	$\tan(x)$	$exp(x)$	$\ln(x)$	Clear
5	Backspace	.	=	Mode shift	π

TABLE VI: Key assignments in Mode 2 with Arduino pin mapping

Column/Row	1	2	3	4	5
1	0	1	2	3	4
2	5	6	7	8	9
3	(	)	x^y	fact(x)	$\sin^{-1}(x)$
4	$\cos^{-1}(x)$	$\tan^{-1}(x)$	mod(x)	$\log_{10}(x)$	Clear
5	Backspace	.	=	Mode shift	π

TABLE VII: Arduino to LCD pin connections

Arduino Pin	**LCD Pin**	**LCD Label**	**Description**
GND	1	GND	Ground
5V	2	Vcc	Power Supply
GND	3	Vee	Contrast Control
D2	4	RS	Register Select
GND	5	R/W	Read/Write (fixed to Write)
D3	6	EN	Enable
D4	11	DB4	Data Bus (4-bit mode)
D5	12	DB5	Data Bus (4-bit mode)
D6	13	DB6	Data Bus (4-bit mode)
D7	14	DB7	Data Bus (4-bit mode)
5V	15	LED+	Backlight Anode
GND	16	LED-	Backlight Cathode

TABLE VIII: Keypad row/column connections to Arduino pins

Keypad Line (Row/Column)	**Arduino Pin**
R1	D8
R2	D9
R3	D10
R4	D11
R5	D12
C1	A0
C2	A1
C3	A2
C4	A3
C5	A4

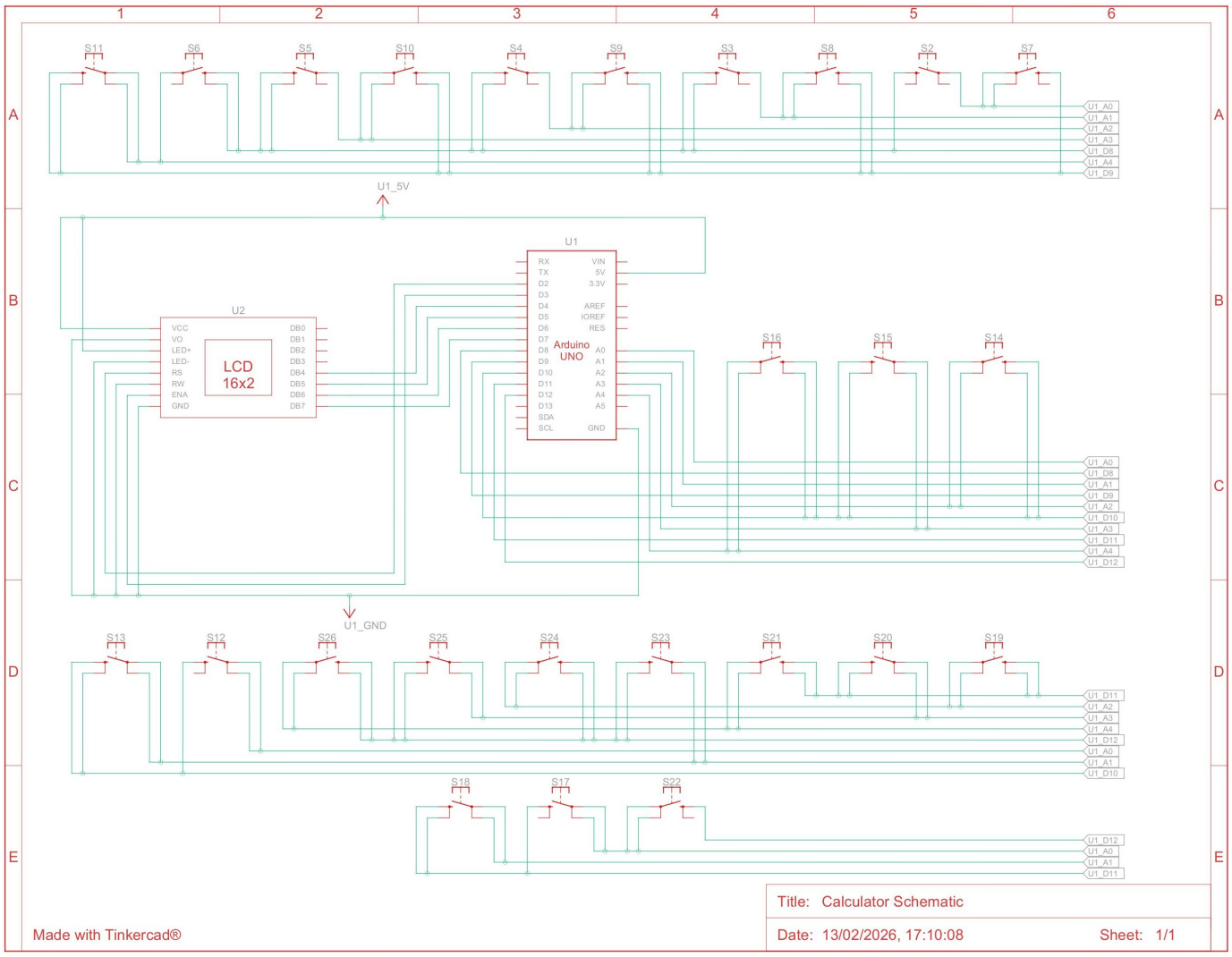
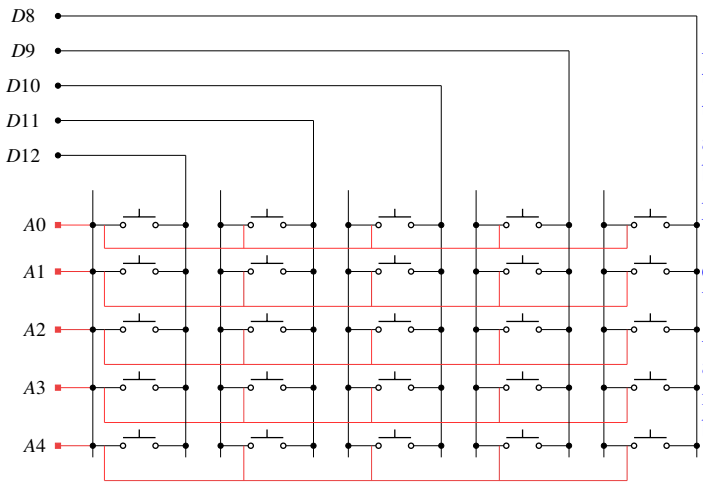


Fig. 2: Schematic of Scientific Calculator

Fig. 1: Button Matrix



IV. Limitations and Hardware Analysis

Table IX details the hardware analysed using codes in Appendix A-B. While the RK4 method has a higher execution time, this trade-off results in reaching the precision floor of the ATMEGA328P. Also, the complete scientific calculator application using RK4 fits in only 12.3 KB of Flash and 350 bytes of RAM, leaving comfortable overhead for additional peripherals or more complex systems.

There were also some limitations due to the hardware that could not be overcome. Lack of a hardware Floating Point Unit required software emulation, adding significant overhead. A single float division costed 505 cycles, more than twice the amount needed for an integer division. This results in inflation in latency per function and a hard precision floor that cannot be overcome with software (see Appendix A-B).

Due to the above reasons, a fixed-step approach had to be used, but this results in both higher latency and error. At the precision floor however, the accuracy is maximised at the cost of latency.

TABLE IX: Hardware Performance Analysis

Algorithm	Flash (Bytes)	SRAM (Bytes)	Execution Time (μs)
RK4	1392	16	192.3125
Trapezoidal	1472	16	101.1875
Forward Euler	1272	16	45.7500
Backward Euler	1296	16	55.5625

Appendix A
Numerical Methods

A. Runge-Kutta Method (RK4)

The RK4 method is a numerical technique to solve

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0 \tag{10}$$

with step size h. The next value is calculated as

$$y_{n+1} = y_n + \frac{1}{6}(m_1 + 2m_2 + 2m_3 + m_4) \tag{11}$$

where m_i are the intermediate slopes [2].

B. Comparative Analysis

The following criteria were considered for choosing an algorithm for a microcontroller base.

- Computing complexity
- Accuracy for the chosen precision level
- Memory usage

The following numerical methods were considered before finalizing the RK4 method.

1) Euler's Method:
   a) Forward Euler (Explicit)

$$y_{n+1} = y_n + h \cdot f(x_n, y_n) \tag{12}$$

   b) Backward Euler (Implicit)

$$y_{n+1} = y_n + h \cdot f(x_{n+1}, y_{n+1}) \tag{13}$$

2) Trapezoidal Rule
   a) Explicit

$$y_{n+1} = y_n + \frac{h}{2} \cdot (f(x_n, y_n) + f(x_{n+1}, y_{n+1})) \tag{14}$$

   b) Digital Filter: Another way to approach the previous formula is tranforming it through the s-domain and z-domain in succession, this reduces the work in solving of ODEs to simple algebraic manipulations. As an example, presented here is the solution to the *sin* function, whose ODE is

$$y'' + y = 0 \tag{15}$$

   i) Laplace Transform: Take the Laplace transform (assuming zero initial conditions):

$$s^2Y(s) + Y(s) = 1 \implies Y(s) = H(s) = \frac{1}{s^2 + 1} \tag{16}$$

   ii) Bilinear Substitution: Apply the Bilinear transform substitution $s \approx \frac{2}{h}\frac{1-z^{-1}}{1+z^{-1}}$, where h is the step, to convert the continuous time transfer function into a discrete time equivalent:

$$H(z) = \frac{1}{\left[\left(\frac{2}{h}\frac{1-z^{-1}}{1+z^{-1}}\right)^2 + 1\right]} \tag{17}$$

   iii) Algebraic Simplification: Let $c = 2/h$, $A = c^2 + 1$. Simplify and group by powers of z

$$H(z) = \frac{(1 + z^{-1})^2}{c^2(1 - z^{-1})^2 + (1 + z^{-1})^2} \tag{18}$$

$$H(z) = \frac{1 + 2z^{-1} + z^{-2}}{c^2(1 - 2z^{-1} + z^{-2}) + (1 + 2z^{-1} + z^{-2})} \tag{19}$$

$$\frac{Y(z)}{X(z)} = \frac{1 + 2z^{-1} + z^{-2}}{(c^2 + 1) + (2 - 2c^2)z^{-1} + (c^2 + 1)z^{-2}} \tag{20}$$

$$Y(z) \cdot \left[1 + \frac{2 - 2c^2}{A}z^{-1} + z^{-2}\right] \tag{21}$$

$$= X(z) \cdot \frac{1}{A}\left[1 + 2z^{-1} + z^{-2}\right] \tag{22}$$

   iv) Difference Equation: Converting from z-domain to discrete time n (where $z^{-k}Y(z) \rightarrow y_{n-k}$ as per inverse Z-transform):

$$y[n] = \underbrace{\frac{1}{A}(x[n] + 2x[n-1] + x[n-2])}_{\text{Input Terms (Source of Energy)}} \tag{23}$$

$$- \underbrace{\frac{2 - 2c^2}{A}y[n-1] - y[n-2]}_{\text{Feedback Terms (Oscillation)}} \tag{24}$$

   Solving for the current term y_n after the impulse has passed:

$$y_n = \frac{8 - 2h^2}{4 + h^2}y_{n-1} - y_{n-2}$$

3) Series Expansion Methods: Taylor/Maclaurin series can be used for this purpose.
4) Second-Order Runge-Kutta Methods (RK2, Heun's Method):

The fixed-step RK4 method was determined to be the optimal choice for this specific hardware implementation based on the following analysis (summarized in Table X). The code for this analysis can be found at: https://github.com/gadepall/calculator/tree/main/analysis Here, each algorithm was compared against the gold-standard MPFR library up to 50 decimal places. Accuracy is represented using Units in Last Place (ULP) measure.

$$\text{Error}_{\text{ULP}} = \frac{|y_{\text{calc}} - y_{\text{true}}|}{\epsilon(y_{\text{true}})} \tag{25}$$

where y_{calc} is the value of the function calculated by the algorithm being tested, y_{true} is the value calculated by the MPFR library, and $\epsilon(y_{\text{true}})$ is the minimum floating point difference representable on a computer between two numbers.

As per the results in Fig.3, RK4 demonstrates a clear

dominance, with both maximum and average errors being many orders of magnitude less than the other algorithms. The Euler methods demonstrate an increasing drift from the true value with increase in input value, while the trapezoidal method, while stable in its error variation, featured much more absolute error than RK4. As expected, RK2 showed similar error variation as RK4 but much greater in magnitude.

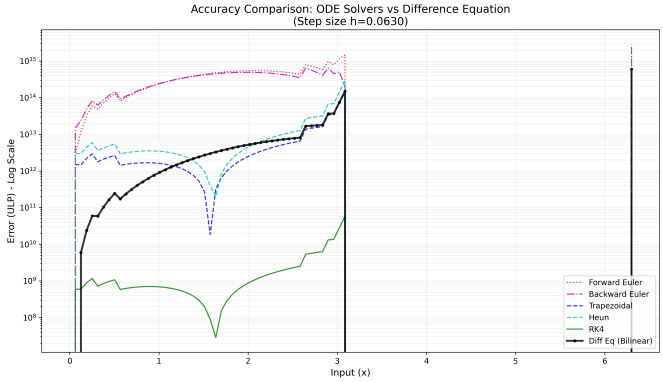


Fig. 3: Comparison of the accuracy of the algorithms

TABLE X: Accuracy comparison for $\sin(x)$ generation ($h \approx 0.06$)

Algorithm	Max Error (ULP)	RMS Error	Stability
RK4	2.38×10^{11}	3.44×10^{-7}	High
Trapezoidal	6.00×10^{14}	8.77×10^{-4}	High
Heun	1.20×10^{15}	1.74×10^{-3}	Moderate
Forward Euler	1.86×10^{15}	8.56×10^{-2}	Unstable
Backward Euler	2.84×10^{15}	7.27×10^{-2}	Unstable

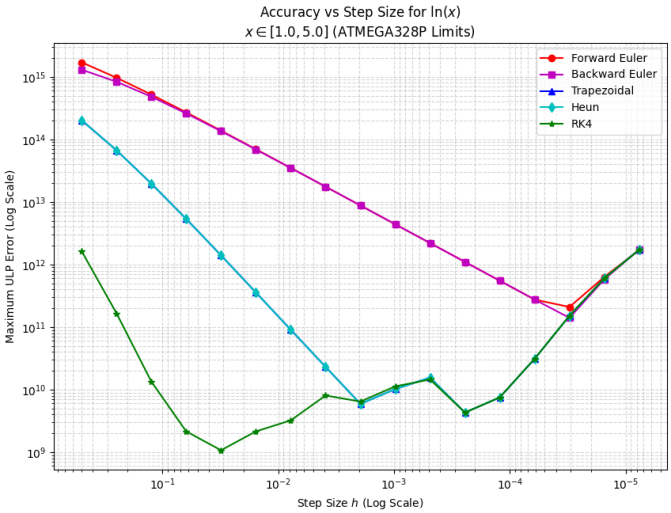


Fig. 4: Accuracy versus step size

Fig. 4 shows an analysis of the variation of error with decrease in step size. While the initial parts of the graph show error as expected from Fig. 3, this plot provides insight into the limitations of the hardware. On 8-bit architectures like on the ATMEGA328P, there is no hardware Floating Point Unit, thus floating point operations are emulated in software. As part of this `avr-gcc` treats `float` and `double` as 32 bit floats. Thus, for RK4, it is seen that there is an error minima occurring at around $h = 0.03$. Using a step size smaller than 0.03 only increases computation time. Also, since RK4 attains its error minima at a relatively high h value, there is no practical use in implementing anything higher than a 4th order algorithm, as the hardware precision floor is the bottleneck. This further cements the objective suitability of RK4 for this purpose.

Appendix B
Software Setup

1) Download the files present in https://github.com/gadepall/calculator/tree/main/software.
2) Compile the C files:

```
gcc -shared -o libcalc.so -fPIC func.c pars.c
```

3) Copy the app APK file `calci.apk` to the phone Internal Storage and install it from the File Manager.
4) Run the server in `termux-debian`

```
python3 server.py
```

5) The app now functions as a scientific calculator.

Appendix C
Hardware Upload

1) Install platformio and create a project in an empty directory

```
pio project init --board uno
```

2) Download the files in https://github.com/gadepall/calculator/tree/main/codes to `src/`.
3) In `platformio.ini`, add the following flags at the end:

```
build_flags = -Wl,-u,vfprintf -lprintf_flt
```

4) Compile and upload.

```
pio run -t upload
```

Table XI describes the various hardware source codes.

File Name	Description
`defines.h`	Global constants and hardware pin mappings.
`func.c/.h`	RK4 solver and implementation of all functions.
`gpio.c/.h`	Interface for GPIO pin control.
`keypad.c/.h`	Logic for initialising and reading keypad input.
`lcd.c/.h`	LCD driver for 16x2 LCD using HD44780 controller interface.
`main.c`	Entry point containing the system init and main loop, as well as handling key presses.
`pars.c/.h`	Parser module for processing input from user and returning output.
`timer.c/.h`	Hardware timer configuration and interface.

TABLE XI: Hardware source code description

References

[1] A. E. Parker, "Runge-kutta 4 (and other numerical methods for ODEs)," Differential Equations, 7, 2021, accessed: 2025-02-09. [Online]. Available: https://digitalcommons.ursinus.edu/triumphs_differ/7

[2] B. S. Grewal, *Higher Engineering Mathematics*, 43rd ed. New Delhi, India: Khanna Publishers, 2014.

[3] N. Ahmad, S. Charan, and V. P. Singh, "Study of numerical accuracy of Runge-Kutta second, third and fourth order method," *International Journal of Computer and Mathematical Sciences (IJCMS)*, vol. 4, no. 6, pp. 111–117, June 2015. [Online]. Available: https://www.researchgate.net/publication/279849033_Study_of_Numerical_Accuracy_of_Runge-Kutta_Second_Third_and_Fourth_Order_Method

[4] K. Murugesan, D. P. Dhayabaran, E. C. H. Amirtharaj, and D. J. Evans, "A fourth order embedded runge-kutta rkacem(4, 4) method based on arithmetic and centroidal means with error control," *International Journal of Computer Mathematics*, vol. 79, no. 2, p. 247–269, Jan. 2002. [Online]. Available: http://dx.doi.org/10.1080/00207160211925

[5] N. A. Z. M. Noar, N. I. A. Apandi, and N. Rosli, "A comparative study of taylor method, fourth order runge-kutta method and runge-kutta fehlberg method to solve ordinary differential equations," in *THE 7TH BIOMEDICAL ENGINEERING'S RECENT PROGRESS IN BIOMATERIALS, DRUGS DEVELOPMENT, AND MEDICAL DEVICES: The 15th Asian Congress on Biotechnology in conjunction with the 7th International Symposium on Biomedical Engineering (ACB-ISBE 2022)*, vol. 3080. AIP Publishing, 2024, p. 020003. [Online]. Available: http://dx.doi.org/10.1063/5.0192085

[6] *AN5325: How to use the CORDIC to perform mathematical functions on STM32 MCUs*, STMicroelectronics, Mar. 2024, rev 6. [Online]. Available: https://www.st.com/resource/en/application_note/an5325-how-to-use-the-cordic-to-perform-mathematical-functions-on-stm32-mcus-stmicroelectronics.pdf

[7] J. P. Allamaa, P. Patrinos, H. Van der Auweraer, and T. D. Son, "Safety envelope for orthogonal collocation methods in embedded optimal control," in *2023 European Control Conference (ECC)*, 2023, pp. 1–7.

[8] A. Akpunar Bozkurt, "Real-time sensorless speed control of pmsms using a runge–kutta extended kalman filter," *Mathematics*, vol. 14, no. 2, p. 274, Jan. 2026. [Online]. Available: http://dx.doi.org/10.3390/math14020274

[9] S. A. Medjahed, "Introduction of the runge-kutta method in gps orbit computation," *International Journal of Engineering and Geosciences*, vol. 9, no. 2, p. 256–263, Jul. 2024. [Online]. Available: http://dx.doi.org/10.26833/ijeg.1403435

[10] L. Moroz, V. Samotyy, P. Gepner, M. Wegrzyn, and G. Nowakowski, "Power function algorithms implemented in microcontrollers and fpgas," *Electronics*, vol. 12, no. 16, p. 3399, Aug. 2023. [Online]. Available: http://dx.doi.org/10.3390/electronics12163399

[11] National Council of Educational Research and Training, *Mathematics: Textbook for Class XII, Part 1 and 2*. New Delhi, India: NCERT, 2006.

[12] G. V. V. Sharma, *C Programming in Middle School*, 1st ed. Hyderabad, India: Narayanpal Prakashan, 2026.

[13] C. Lomont, "Fast inverse square root," Purdue University, Technical Report, February 2003. [Online]. Available: http://www.lomont.org/papers/2003/InvSqrt.pdf

[14] E. W. Dijkstra, "Algol 60 translation: An Algol 60 translator for the X1 and Making a translator for Algol 60," Mathematisch Centrum, Amsterdam, Tech. Rep. MR 35/61, November 1961. [Online]. Available: http://www.cs.utexas.edu/users/EWD/ewd00xx/EWD35.PDF